commons-cli-scope.txt
TXT 149.65KB

You are a code reviewer. The attached file "commons-cli-scope.txt" contains the Java source code of the Apache Commons CLI library (34 files, comments removed). Each file starts with a line "// ===== FILE: <path> =====".

Analyze the code for exactly these three code smells and ignore all others:

# Code Smell Analysis: Apache Commons CLI

## 1. Long Method

**Smell:** Long Method
**Code unit:** `DefaultParser.handleShortAndLongOption(String)`
**File:** `org/apache/commons/cli/DefaultParser.java`
**Evidence:** The method body spans ~55 lines with a multi-branch `if/else if/else` chain: `if (token.length() == 1)`, `else if (pos == -1)` containing nested branches for `hasShortOption`, `getMatchingLongOptions`, `getLongPrefix`, `isJavaProperty`, and `handleConcatenatedOptions`; then an `else` with `if (opt.length() == 1)`, `else if (isJavaProperty(opt))`, `else handleLongOptionWithEqual`. Each branch calls multiple helpers and manipulates `currentOption`.
**Explanation:** The method performs token classification, short-option handling, long-option handling, long-prefix handling, Java-property handling, concatenated-option handling, and value processing all in one body, making it hard to follow and maintain.

**Smell:** Long Method
**Code unit:** `DefaultParser.handleToken(String)`
**File:** `org/apache/commons/cli/DefaultParser.java`
**Evidence:** A chain of `if/else if` branches: `if (skipParsing)`, `else if (OptionFormatter.DEFAULT_LONG_OPT_PREFIX.equals(token))`, `else if (currentOption != null && currentOption.acceptsArg() && isArgument(token))`, `else if (token.startsWith(...DEFAULT_LONG_OPT_PREFIX))`, `else if (token.startsWith(...DEFAULT_OPT_PREFIX) && ...)`, `else`, followed by a second

`if (currentOption != null && !currentOption.acceptsArg())` .

**Explanation:** This method handles all token dispatch (skip mode, bare `--` , argument consumption, long options, short options, unknown tokens) plus post-processing, doing too many things in one place.

**Smell:** Long Method

**Code unit:** `DefaultParser.parse(Options, Properties, NonOptionAction, String...)`

**File:** `org/apache/commons/cli/DefaultParser.java`

**Evidence:** The method initializes fields ( `this.options` , `this.nonOptionAction` , `skipParsing` , `currentOption` , `expectedOpts` ), resets all option groups ( `optionGroup.setSelected(null)` ), builds the `CommandLine` , loops over arguments calling `handleToken` , then calls `checkRequiredArgs()` , `handleProperties(properties)` , and `checkRequiredOptions()` .

**Explanation:** It combines state initialization, group resetting, command-line construction, argument iteration, property handling, and validation in a single method.

**Smell:** Long Method

**Code unit:** `HelpFormatter.appendOptions(A, int, Options, int, int)`

**File:** `org/apache/commons/cli/HelpFormatter.java`

**Evidence:** The method computes `max` and `maxSince` , builds a `prefixList` in a loop (with branches for `getOpt() == null` , `hasLongOpt()` , `hasArg()` , empty `argName` ), optionally emits a header ( `showSince` block building `optHeader` ), then a second loop renders each option with padding logic, `deprecatedFormatFunction` branching, and calls `appendWrappedText` .

**Explanation:** It performs column-width measurement, header generation, per-option prefix construction, and wrapped rendering in one long body with multiple loops and branches.

**Smell:** Long Method

**Code unit:** `CommandLine.getParsedOptionValues(Option, Supplier<T[]>)`

**File:** `org/apache/commons/cli/CommandLine.java`

**Evidence:** It handles a null `option` , casts `option.getType()` to `Class<? extends T>` , fetches `getOptionValues(option)` , allocates `Array.newInstance(clazz, values.length)` , loops converting each value via `option.getConverter().apply(...)` , and wraps exceptions with `ParseException.wrap(t)` .

**Explanation:** Null/default handling, type extraction, array allocation, per-element conversion, and exception wrapping are combined into a single method.

**Smell:** Long Method

**Code unit:** `TypeHandler.putDefaultMap(Map)`

**File:** `org/apache/commons/cli/TypeHandler.java`

**Evidence:** A long sequence of `map.put(...)` calls covering `Object`, `Class`, `Date`, `File`, `Path`, `Number`, `URL`, `FileInputStream`, `Long`, `Integer`, `Short`, `Byte`, `Character` (with a multi-line lambda handling `\\u` code points and surrogate checks), `Double`, `Float`, `BigInteger`, `BigDecimal`.

**Explanation:** The method is a large registry-population routine that also embeds non-trivial `Character` parsing logic inline, making it lengthy and doing more than a simple map setup.

**Smell:** Long Method

**Code unit:** `TextHelpAppendable.adjustTableFormat(TableDefinition)`

**File:** `org/apache/commons/cli/help/TextHelpAppendable.java`

**Evidence:** It builds `styleBuilders` in a loop adjusting `maxWidth` / `minWidth` against headers and every row cell, then a second loop accumulates `calcWidth` / `adjustedMaxWidth`, computes a `fraction`, a third loop resizes scalable builders, and finally assembles a new `TableDefinition`.

**Explanation:** Width measurement, scaling computation, and rebuild are all combined in one method with three loops and multiple mutable accumulators.

**Smell:** Long Method

**Code unit:** `PatternOptionBuilder.parsePattern(String)`

**File:** `org/apache/commons/cli/PatternOptionBuilder.java`

**Evidence:** It iterates pattern chars, branches on `isValueCode(ch)`, `ch == '!'`, `getValueType(ch)`, creates a `TypeHandler` with `createDefaultMap()`, `map.put(FILES_VALUE, unsupported())`, then builds options in two places (inside the loop when `opt != Char.SP`, and after the loop).

**Explanation:** Character scanning, required-flag handling, type mapping, converter construction, and option creation are all performed in one method body.

**Smell:** Long Method

**Code unit:**

`HelpFormatter.printHelp(PrintWriter, int, String, String, Options, int, int, String, boolean)`

**File:** `org/apache/commons/cli/HelpFormatter.java`

**Evidence:** Validates `cmdLineSyntax`, branches on `autoUsage` to call `printUsage(pw, width, cmdLineSyntax, options)` vs `printUsage(pw, width, cmdLineSyntax)`

, conditionally prints header, calls `printOptions`, and conditionally prints footer.
**Explanation:** It orchestrates the entire help-output pipeline (usage, header, options, footer) with several conditional branches in one method.

## 2. God Class

**Smell:** God Class
**Code unit:** `CommandLine`
**File:** `org/apache/commons/cli/CommandLine.java`
**Evidence:** Over 60 public methods covering args (`getArgList`, `getArgs`), option counting (`getOptionCount` ×3), deprecated object access (`getOptionObject` ×2), properties (`getOptionProperties` ×2), option arrays (`getOptions`), value retrieval (`getOptionValue` ~15 overloads across `char`, `Option`, `OptionGroup`, `String`), value arrays (`getOptionValues` ×4), parsed values (`getParsedOptionValue` ~14 overloads), parsed value arrays (`getParsedOptionValues` ~14 overloads), presence checks (`hasOption` ×4), iteration (`iterator`), plus `resolveOption`, `processPropertiesFromValues`, `handleDeprecated`, and a nested `Builder`.
**Explanation:** The class accumulates option lookup, value parsing, type conversion, property extraction, deprecation handling, and iteration responsibilities with an enormous method surface, indicating it knows and does far too much.

**Smell:** God Class
**Code unit:** `DefaultParser`
**File:** `org/apache/commons/cli/DefaultParser.java`
**Evidence:** Fields `cmd`, `options`, `stopAtNonOption`, `nonOptionAction`, `currentToken`, `currentOption`, `skipParsing`, `expectedOpts`, `allowPartialMatching`, `stripLeadingAndTrailingQuotes`, `deprecatedHandler`; methods for prefix matching (`getLongPrefix`, `getMatchingLongOptions`), long-option handling (`handleLongOption`, `handleLongOptionWithEqual`, `handleLongOptionWithoutEqual`), concatenation (`handleConcatenatedOptions`), short/long dispatch (`handleShortAndLongOption`), token dispatch (`handleToken`), unknown tokens (`handleUnknownToken`), classification (`isArgument`, `isJavaProperty`, `isLongOption`, `isNegativeNumber`, `isOption`, `isShortOption`), option updates (`handleOption`, `updateRequiredOptions`), properties (`handleProperties`), validation (`checkRequiredArgs`, `checkRequiredOptions`), quote stripping, plus nested `Builder` and `NonOptionAction` enum.
**Explanation:** It owns parsing state, token classification, option matching,

group/required-option bookkeeping, property handling, and deprecation forwarding, coupling it heavily to `Options`, `Option`, `OptionGroup`, `CommandLine`, and `Properties`.

**Smell:** God Class
**Code unit:** `HelpFormatter`
**File:** `org/apache/commons/cli/HelpFormatter.java`
**Evidence:** Public configuration fields (`defaultWidth`, `defaultLeftPad`, `defaultDescPad`, `defaultSyntaxPrefix`, `defaultNewLine`, `defaultOptPrefix`, `defaultLongOptPrefix`, `defaultArgName`), plus ~40 methods: `appendOption`, `appendOptionGroup`, `appendOptions`, `appendWrappedText`, `createPadding`, `determineMaxSinceLength`, `findWrapPos`, all getters/setters (`getArgName`, `getDescPadding`, `getLeftPadding`, `getLongOptPrefix`, `getLongOptSeparator`, `getNewLine`, `getOptionComparator`, `getOptPrefix`, `getSyntaxPrefix`, `getWidth`, and matching setters), `printHelp` ×6, `printOptions`, `printUsage` ×2, `printWrapped` ×2, `renderOptions`, `renderWrappedText`, `renderWrappedTextBlock`, `rtrim`, nested `Builder` and `OptionComparator`.
**Explanation:** It combines text wrapping, padding, syntax/usage rendering, option-group rendering, option-comparator handling, and output configuration, with wide-reaching getters/setters exposing internal layout data.

**Smell:** God Class
**Code unit:** `Option`
**File:** `org/apache/commons/cli/Option.java`
**Evidence:** Fields `argCount`, `argName`, `converter`, `deprecated`, `description`, `longOption`, `option`, `optionalArg`, `required`, `since`, `type`, `values`, `valueSeparator`; methods spanning value storage (`add`, `addValue`, `clearValues`, `processValue`, `getValue` ×3, `getValues`, `getValuesList`), argument acceptance (`acceptsArg`, `requiresArg`), predicates (`hasArg`, `hasArgName`, `hasArgs`, `hasLongOpt`, `hasOptionalArg`, `hasValueSeparator`, `isDeprecated`, `isRequired`, `isValuesEmpty`), metadata getters/setters (`getArgName`, `getArgs`, `getConverter`, `getDeprecated`, `getDescription`, `getId`, `getKey`, `getLongOpt`, `getOpt`, `getSince`, `getType`, `setArgName`, `setArgs`, `setConverter`, `setDescription`, `setLongOpt`, `setOptionalArg`, `setRequired`, `setType` ×2, `setValueSeparator`), `clone`, `equals`, `hashCode`, `toDeprecatedString`, `toString`, plus a full nested `Builder`.
**Explanation:** It blends option metadata, value-list management, type/converter configuration, deprecation attributes, serialization, and a builder, making it a data-heavy class with many responsibilities.

**Smell:** God Class

**Code unit:** `TextHelpAppendable`

**File:** `org/apache/commons/cli/help/TextHelpAppendable.java`

**Evidence:** Methods `indexOfWrap` (static), `systemOut`, `adjustTableFormat`, `appendHeader`, `appendList`, `appendParagraph`, `appendTable`, `appendTitle`, `getIndent`, `getLeftPad`, `getMaxWidth`, `getTextStyleBuilder`, `makeColumnQueue`, `makeColumnQueues`, `printQueue`, `printWrapped` ×2, `resize` ×2, `setIndent`, `setLeftPad`, `setMaxWidth`, `writeColumnQueues`.

**Explanation:** It handles line wrapping, table width scaling, header/title/list/table rendering, column-queue construction, and layout configuration, aggregating many rendering responsibilities.

## 3. Long Parameter List

**Smell:** Long Parameter List

**Code unit:**

`HelpFormatter.printHelp(PrintWriter, int, String, String, Options, int, int, String, boolean)`

**File:** `org/apache/commons/cli/HelpFormatter.java`

**Evidence:** Signature:

`printHelp(final PrintWriter pw, final int width, final String cmdLineSyntax, final String header, final Options options, final int leftPad, final int descPad, final String footer, final boolean autoUsage)`

— 9 parameters.

**Explanation:** Nine parameters (output, width, syntax, header, options, two paddings, footer, flag) exceed a comfortable threshold and bundle layout configuration into the call site.

**Smell:** Long Parameter List

**Code unit:**

`HelpFormatter.printHelp(PrintWriter, int, String, String, Options, int, int, String)`

**File:** `org/apache/commons/cli/HelpFormatter.java`

**Evidence:** Signature:

`printHelp(final PrintWriter pw, final int width, final String cmdLineSyntax, final String header, final Options options, final int leftPad, final int descPad, final String footer)`

— 8 parameters.

**Explanation:** Eight parameters carry output destination, dimensions, text sections, options, and padding values in one signature.

**Smell:** Long Parameter List

**Code unit:** `HelpFormatter.printHelp(int, String, String, Options, String, boolean)`

**File:** `org/apache/commons/cli/HelpFormatter.java`

**Evidence:** Signature:

```
printHelp(final int width, final String cmdLineSyntax, final String header, final
Options options, final String footer, final boolean autoUsage)
```

— 6 parameters.

**Explanation:** Six parameters combine width, three text elements, options, and a usage flag, making the call hard to read.

**Smell:** Long Parameter List

**Code unit:** `HelpFormatter.printHelp(String, String, Options, String, boolean)`

**File:** `org/apache/commons/cli/HelpFormatter.java`

**Evidence:** Signature:

```
printHelp(final String cmdLineSyntax, final String header, final Options options,
final String footer, final boolean autoUsage)
```

— 5 parameters.

**Explanation:** Five parameters (syntax, header, options, footer, flag) bundle the complete help layout into a single call.

**Smell:** Long Parameter List

**Code unit:** `DefaultParser.parse(Options, Properties, NonOptionAction, String...)`

**File:** `org/apache/commons/cli/DefaultParser.java`

**Evidence:** Signature:

```
parse(final Options options, final Properties properties, final NonOptionAction
nonOptionAction, final String... arguments)
```

.

**Explanation:** It mixes options, defaults, a behavioral action, and the argument array in one signature, and is the funnel through which all `parse` overloads route.

**Smell:** Long Parameter List

**Code unit:** `Option.Option(String, String, boolean, String)`

**File:** `org/apache/commons/cli/Option.java`

**Evidence:** Signature:

```
Option(final String option, final String longOption, final boolean hasArg, final
String description)
```

.

**Explanation:** Four constructor parameters combine short name, long name, argument flag, and description, forcing callers to remember positional order.

**Smell:** Long Parameter List

**Code unit:**

`TableDefinition.from(String, List<TextStyle>, List<String>, Iterable<List<String>>)`

**File:** `org/apache/commons/cli/help/TableDefinition.java`

**Evidence:** Signature:

`from(final String caption, final List<TextStyle> columnStyle, final List<String> headers, final Iterable<List<String>> rows)`

.

**Explanation:** Four parallel collection/string parameters describe one logical table and must be supplied in the correct order.

**Smell:** Long Parameter List

**Code unit:** `OptionFormatter.Builder.setArgumentNameDelimiters(String, String)` / `setOptionalDelimiters(String, String)`

**File:** `org/apache/commons/cli/help/OptionFormatter.java`

**Evidence:** Both take a begin/end pair:

`setArgumentNameDelimiters(final String begin, final String end)` and `setOptionalDelimiters(final String begin, final String end)` .

**Explanation:** While only two parameters, they are paired delimiters whose positional meaning is easy to confuse; included as a borderline case of parameter-pair coupling. (Confidence lower than the others above.)

## Summary

- **Long Method:** 9 detections (concentrated in `DefaultParser` , `HelpFormatter` , and the help/text rendering classes).
- **God Class:** 5 detections ( `CommandLine` , `DefaultParser` , `HelpFormatter` , `Option` , `TextHelpAppendable` ).
- **Long Parameter List:** 8 detections (primarily `HelpFormatter.printHelp` overloads, plus `DefaultParser.parse` , `Option` constructor, `TableDefinition.from` , and the `OptionFormatter.Builder` delimiter setters).